

2026년 오픈소스 개발자대회 결과보고서

항 목	내 용	항 목	내 용
팀 명	[팀명]	팀 인 원 (팀장 포함)	1명
참가부문	일반	과제유형	지정과제(SK텔레콤)

□ 결과보고서

프로젝트 개요	
프로젝트명	프롬프트 난이도 인지 기반 경량 LLM 라우터
프로젝트 등록 URL	https://github.com/leelang7/ossdp-2026-llm-router-challenge/tree/[최종 커밋 SHA] ※ submission-ossdp-skt.json을 포함한 커밋의 스냅샷 URL
시연영상	[유튜브 URL — 업로드 후 기재]
프로젝트 소개	프롬프트 내용만 보고 세 후보 모델 중 하나를 골라 정해진 예산 안에서 답변 품질을 최대화하는 오픈소스 LLM 라우터. 모델을 호출하지 않고 네트워크도 쓰지 않으며, 학습된 계수와 어휘만으로 문항당 약 5밀리초에 판단한다.
프로젝트 세부 내용	
개발배경 및 목적	○ LLM 서비스 운영비의 대부분은 추론 비용이지만, 실제 트래픽의 상당수는 저렴한 모델로도 충분한 단순 질의다. 그럼에도 많은 서비스가 모든 요청을 고성능 모델로 보내 예산을 낭비한다. ○ 이 프로젝트는 '쉬운 질문은 싸게, 어려운 질문만 비싸게'를 자동으로 판단하는 라우터를 오픈소스로 구현해, 같은 예산에서 더 높은 평균 품질을 얻는 것을 목표로 한다. ○ 과제의 제약은 실제 서비스 환경을 반영한다. 라우터는 모델을 호출할 수 없고 답변을 비교하거나 선택을 반복할 수도 없다. 프롬프트와 예산 등급만 보고 한 번에 정해야 하며, 예산을 넘기면 해당 등급은 0점이다. 운영에서 대기열 폭증이나 응답 지연으로 이어지는 상황을 그대로 옮긴 조건이다.
개발환경	○ 언어·런타임 : Python 3.11, 컨테이너 linux/arm64 (Debian slim 기반) ○ 학습 환경 : scikit-learn 1.8.0, SciPy 1.17.1, NumPy 1.26.4 (로컬 CPU) ○ 실행 의존성 : NumPy 하나 — 학습용 패키지는 제출 이미지에 넣지

	없음 ○ 공식 실행 조건 : CPU 2코어, 메모리 2GiB, 네트워크 없음, 읽기 전용 루트, 등급당 90초, 비트코인 사용자(UID 65532) ○ 개발 도구 : Git, Docker Buildx, 표준 라이브러리 unittest ○ 산출물 크기 : 컨테이너 이미지 73.4MB, 학습 아티팩트 5.5MB
시스템 구성 및 아키텍처	○ 특징 추출 (프롬프트 내용만 사용) - 직접 계산 특징 36개 : 길이, 한글 비율, 코드·수식 표지, 객관식 표지, 대화 구조 등 - TF-IDF 단어 1~2그램 (6만 차원), 문자 3~5그램 (12만 차원) ○ 예측 (ridge 회귀 7개 헤드) - 모델별 예측 품질 3개 - 모델별 예측 출력 토큰 3개, 예측 입력 토큰 1개 → 공개 비용 정책으로 비용 환산 ○ 배치 선택 정책 - 예측 이득 ÷ 예측 비용이 큰 순서로 승격 - 동률은 프롬프트 내용 해시로만 정렬 (문항 ID·입력 순서 미사용) - 등급별 예산 마진, 추론 모델 선택 건수 상한, 문항별 비용 상한 적용 ○ 출력 : 등급별 선택 결과 JSON (문항당 model_id 하나)
프로젝트 주요기능	① 비용을 예측 대상에 포함 라우팅 시점에는 토큰 수가 주어지지 않는다. 그래서 품질뿐 아니라 모델별 출력 토큰과 입력 토큰까지 회귀로 예측하고 공개 비용 정책으로 환산한다. 학습에서도 실제 토큰 수 대신 예측값으로 비용을 계산해 추론과 조건을 맞췄다. 로그 공간 회귀를 지수로 되돌리면 평균이 과소평가되므로(Jensen 부등식) 잔차 분위수만큼 상향한 뒤, 학습 분할의 실제 총비용에 맞도록 모델별 스케일을 보정한다. ② 예산 초과를 구조로 차단 추론 모델(axk1-think)은 출력 토큰 분포의 꼬리가 매우 두껍다. 공개 자료에서 중앙값이 1,570토큰인데 최대는 130,504토큰으로, 단 한 문항이 경량 모델 전체 비용의 26%를 소모할 수 있다. 몇 건만 빚나가도 등급이 통째로 0점이 된다. 문항별 예측 비용 상한과 분위수 회귀는 모두 실패했다. 사고를 내는 문항은 '예측은 작는데 실재가 큰' 것들이라 예측값에 의존하는 필터를 그대로 통과하기 때문이다. 효과가 있었던 것은 추론 모델의 선택 건수 자체를 묶는 것이었다. 예측 오차와 무관하게 노출이 제한되며, 고비용 구간은 승격 이득 대비 비용도 가장 나쁜 구간이라 점수 손실도 작다.

③ 순서에 의존하지 않는 배치 정책

예산은 배치 전체에 걸린 제약이라 문항을 독립적으로 처리할 수 없다. 그렇다고 입력 순서에 의존하면 규칙 위반이다. 승격 후보를 이득/비용비로 정렬하되 동률은 프롬프트 내용의 SHA-256 해시로만 깨고, 내용이 같은 문항은 그룹으로 묶어 통째로 승격한다. 운영자가 순서를 섞고 문항 ID를 바꿔 재실행해도 결과가 같다.

④ 학습과 실행의 분리

학습은 scikit-learn으로 하되 제출 이미지에는 NumPy만 넣는다. TfidfVectorizer의 동작(소문자화, 토큰 패턴, char_wb 패딩, sublinear tf, smooth idf, 블록별 L2 정규화)을 표준 라이브러리로 재현했고, 두 경로의 예측 차이는 $1.2e-15$ 로 부동소수점 한계까지 일치한다.

⑤ 등급별 설정은 교차검증으로 결정

2,640문항을 5-fold와 8-fold로 나눠 등급별 (추론모델 상한, 예산 마진) 조합을 탐색하고, 양쪽 fold 구성에서 예산 초과가 0건인 설정만 후보로 삼았다.

- fast : 추론모델 0%, 마진 0.85 (Dev 예산 사용률 82.9%)
- balanced : 추론모델 1%, 마진 0.80 (80.1%)
- premium : 추론모델 11%, 마진 0.85 (64.7%)

⑥ 구동 및 시연

학습은 train_routerx/train.py, 라우팅은 routerx.cli, 자체 점검은 routerx.audit 명령으로 수행한다. 컨테이너는 공식 자원 한도(CPU 2 코어, 2GiB, 네트워크 없음, 읽기 전용 루트)에서 실행을 확인했다.

[자체 점검 결과 — 공개 Dev 880문항]

- ID·순서 불변성 : 400문항 불일치 0건
- 결정성(동일 입력 재실행) : 300문항 불일치 0건
- 엣지 케이스 7종 통과 (빈 프롬프트, 초장문, 어휘 밖 문자, 이모지, 긴 대화 등)
- 실행 시간 : 등급당 4.2초 = 한도 90초의 5% (약 4.8ms/문항)
- 예산 : 세 등급 모두 통과, 여유 17~35%
- 단위 시험 21건 통과

[성능]

- 공개 Dev 880문항 : 최종 0.7167
- 교차검증 2,640문항 : 0.6643 (5-fold-8-fold, 예산 초과 0건)
- 공식 최강 baseline(hash-regex) : 0.6954 / 전부 경량 모델 : 0.6193

기대효과 및 활용분야	○ 모델군에 묶이지 않는다. 후보 모델의 품질·토큰 사용량 기록과 비용 정책만 있으면 어떤 조합에도 재학습할 수 있고, 학습 스크립트와 평가 도구를 함께 공개하므로 사내 모델 구성에 맞춘 재현이 가능하다. ○ 네트워크와 GPU 없이 CPU 2코어에서 동작하므로 사내망·온프레미스·공공 환경에 그대로 넣을 수 있다. 아티팩트가 5.5MB라 엡지 환경에도 실린다. ○ 대규모 트래픽을 다루는 고객센터·검색·공공 AI 서비스에서 같은 예산으로 더 나은 품질을, 또는 같은 품질을 더 낮은 비용으로 제공하는 데 쓸 수 있다. ○ 라우팅 정책·학습 파이프라인·평가 도구를 모두 공개해 LLM 인프라 비용 최적화 연구의 재현 가능한 기준점이 된다.
기타	[혁신성 및 차별성] ① 예산 초과를 예측 정확도가 아니라 구조로 막는다. 예측 기반 방어책이 실패하는 이유를 실험으로 규명하고, 선택 건수 상한이라는 예측 무관 장치로 대체했다. ② 순서 불변성을 설계 단계에서 보장한다. 배치 최적화를 하면서도 동률 정렬 키를 프롬프트 내용 해시로 두어, 감사 항목을 사후 확인이 아니라 구조로 만족한다. ③ 학습·실행 경로 일치를 수치로 증명한다. 두 경로의 예측 차이 $1.2e-15$를 시험으로 고정해, 라이브러리 없는 실행 환경에서도 학습 결과가 재현됨을 보장한다. ④ 제출 전 자체 점검 도구를 함께 제공한다. 규칙 준수·결정성·엡지 케이스·시간·예산을 한 명령으로 확인하고 예산이 한도의 97%를 넘으면 경고한다. ⑤ 도달 가능 상한을 분석해 개발 방향을 정했다. 정보 수준별 도달 점수(전부 경량 0.605 / 비용만 정확 0.683 / 추론모델 이득 완전정보 0.725 / 완전정보 0.792)를 계산해 남은 개선 여지를 정량화했다. ⑥ 측정에 근거해 두 번 되돌렸다. 부스팅 트리는 토큰 예측 상관을 0.37에서 0.65로 올렸고 단일 측정 점수도 가장 높았지만 교차검증에서 예산을 지키지 못해 채택하지 않았다. 문장 임베딩은 오히려 예측을 떨어뜨렸다(0.425→0.372). [한계점 및 향후 로드맵] ○ 경량 모델과 중형 모델의 품질 차이는 예측 상관이 0.01~0.10으로 사실상 예측되지 않는다. 점수가 생성 2~4회 평균이라 노이즈가 지배적이기 때문이며, 부스팅 트리나 다국어 임베딩으로도 넘지 못했다. 반복 횟수가 늘어난 자료가 생기면 개선 여지가 있다. ○ 안전을 위해 예산의 17~35%를 남긴다. 출력 토큰 예측 정확도를

높이면 같은 안전 수준에서 더 쓸 수 있다.

○ 로드맵 : (1) 출력 길이 예측 강화로 예산 활용률 개선 (2) 실시간 서빙용 온라인 예산 컨트롤러 (3) 다른 모델군·비용 정책으로의 이식 사례 추가

[소감 및 후기]

가장 많은 시간을 쓴 것은 점수를 올리는 일이 아니라 0점을 피하는 일이었다. 예산을 한도까지 채우는 정책은 공개 자료에서 잘 작동하다가 분할을 바꾸면 초과했고, 그때마다 등급 점수가 통째로 사라졌다. 평균을 조절하는 마진으로는 두꺼운 꼬리를 막지 못한다는 것을 여러 실패 끝에 확인하고 나서야, 위험을 예측하려 들지 말고 노출 자체를 묶는 방향으로 바꿀 수 있었다. 학습 코드와 실행 코드가 갈라진 지점에서 예측이 어긋난 결함도 뼈아팠다. 공식 검증은 통과했지만 실제로는 세 등급 모두 예산을 넘기는 상태였고, 자체 점검 도구를 만들지 않았다면 제출 후에야 알았을 것이다. 검증을 코드로 만들어 두는 일이 기능 개발만큼 중요하다는 것을 체감했다.

붙임1

SBOM[소프트웨어 자재명세서]

번호	라이브러리명	버전	라이선스	공식 저장소 URL(GitHub 등)	사용 목적 및 주요 기능
1	NumPy	1.26.4	BSD-3-Clause	https://github.com/numpy/numpy	실행 의존성. 계수 행렬 연산과 배치 선택 계산
2	scikit-learn	1.8.0	BSD-3-Clause	https://github.com/scikit-learn/scikit-learn	학습 전용. TF-IDF 어휘·IDF 생성, ridge 회귀 학습
3	SciPy	1.17.1	BSD-3-Clause	https://github.com/scipy/scipy	학습 전용. 희소 행렬 연산
4	PyArrow	23.0.1	Apache-2.0	https://github.com/apache/arrow	자료 준비 전용. 공개 Train/Dev 원본 변환
5	CPython	3.11	PSF-2.0	https://github.com/python/cpython	실행 런타임
6	python:3.11-slim-bookworm	bookworm	다중(GPL/LGPL/MIT/BSD 등)	https://hub.docker.com/_/python	컨테이너 기반 이미지. 상업적 이용·재배포 허용
7	OpenBLAS / libgfortran 등	NumPy 휠 번들	BSD-3-Clause / GPL-3.0-with-GCC-exception	https://github.com/numpy/numpy	NumPy manylinux 휠에 포함된 네이티브 라이브러리
8	ossp_router (과제 제공)	v1	Apache-2.0	https://github.com/sktelecom/ossp-2026-llm-router-challenge	입출력 규격·채점 도구(과제 제공 하네스)

※ 필요 시, 행을 추가하여 기재해 주시기 바랍니다.

붙임2

AI 모델 활용 및 라이선스 기술 명세서

[작성 안내]

- 본 서식은 프로젝트에 탑재·적용된 AI 모델의 오픈소스 요건 및 라이선스 준수 여부를 검증하기 위한 기술 명세서입니다.
- 참가팀은 본인의 AI 개발 유형(유형1, 2, 3)을 먼저 선택한 후 해당하는 항목의 명세를 정확히 작성하여 제출해 주시기 바랍니다.

1. AI 모델 활용 유형 (해당하는 항목에 ☒ 표시)

- ☐ 유형 1: 외부 모델 그대로 활용 (추가 학습 없이 기존 공개 모델을 프로젝트에 연동·구동한 경우)
- ☐ 유형 2: 외부 모델 파인튜닝 (기존 공개 모델을 가져와 준비한 데이터셋으로 추가 미세조정된 경우)
- ☒ 유형 3: 자체 개발 모델 (기반 모델 없이 참가팀이 처음부터 가중치를 직접 전체 학습시킨 경우)
- ※ 개발 과정에서 코딩 및 디버깅 보조용으로 상용 AI(GPT, Claude 등)를 단순 활용한 경우는 체크하지 않음(4번 항목에 기재)

2. 기반(베이스) 모델 정보 (유형 1, 유형 2 작성 필수 / 유형 3은 '해당 없음'으로 기재)

기반 모델명 및 개발사	해당 없음 (기반 모델 없이 처음부터 학습)	기반 모델 라이선스	해당 없음
--------------	--------------------------	------------	-------

3. 데이터셋 정보 및 가중치 배포 명세 (유형 2, 유형 3 작성 필수)

학습 데이터셋 정보 (출처 및 규모)	과제 제공 공개 Train 1,760문항 + Dev 880문항 (총 2,640문항). 프롬프트와 모델별 품질·토큰 사용량, 출처와 라이선스는 과제 저장소 THIRD_PARTY_NOTICES.md에 공개되어 있다.
데이터 정제/ 가공 방법 요약	과제가 제공한 materialize_public_data.py 절차를 그대로 사용했다. 개인정보가 포함되지 않은 공개 벤치마크 자료이므로 별도 비식별화 대상이 없다. 텍스트는 소문자화와 토큰화만 수행하고 원문을 변형하지 않았다.
새로 생성된 가중치 공개 저장소 URL	https://github.com/leelang7/oss-2026-llm-router-challenge 의 src/routerx/artifact.npz (승인 절차 없이 누구나 접근 가능, 수상 시 5년간 공개 유지)
가중치 파일 정보 및 배포방식	파일명 : artifact.npz / 용량 5.5MB / 저장소에 직접 커밋 내용 : ridge 회귀 계수, TF-IDF 어휘·IDF, 특징 정규화 통계, 등급별 안전계수와 상한값

4. 소스코드 라이선스 및 개발 환경 정보 (모든 유형 필수 작성)

직접 작성한 코드의 오픈소스 라이선스	Apache License 2.0	학습/추론 소스코드 공개 저장소 URL	https://github.com/leelang7/oss-2026-llm-router-challenge (공개 유지 조건 준수)
상용 AI 보조도구 활용 여부 및 범위	코드 작성·실험 설계·디버깅 보조에 Claude(Anthropic)를 활용했다. 알고리즘 설계와 실험 결과 해석, 채택 여부의 최종 판단은 참가자가 수행했다.		